Reviewer Pack — Minimal Rank of Quantum Logics and Resolution Proof Width

Entry 559c6c08bf2f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 17:29:11 UTC

Minimal Rank of Quantum Logics and Resolution Proof Width

Entry ID: `559c6c08bf2f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 17:29:11 UTC

1. Conjecture

Field A (mathematical branch): Quantum Logic **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every instance φ of a CNF, the minimal rank of its associated quantum logic L(φ) is linearly correlated with its resolution proof width w(φ), such that rank(L(φ)) = Θ(w(φ)).

Rationale (proposer's reasoning):

> Quantum logics introduce non-classical structure to classical logic. Investigating their ranks may reveal hidden complexity relationships in resolution proofs, potentially leading to new proof systems or lower bounds.

Taxonomy category: `Quantum Logic × Resolution Proof Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `24dc648839fc11ff`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the 1200 CNF instances (n ≤ 40) demonstrate a correlation coefficient r ≥ 0.7 between the minimal rank of the associated quantum logic L(φ) and its resolution proof width w(φ), with no seed producing an r < 0.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: `ADJACENT_OK` against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `quantum logic AND resolution proof complexity` - `minimal rank quantum logic AND resolution proof width` - `θ(w(φ)) correlation with rank(L(φ)) in CNF`

Top relevant hits considered: - [http://arxiv.org/abs/1108.6261v2] The complexity of admissible rules of Łukasiewicz logic - [http://arxiv.org/abs/cs/0404023v2] Propositional computability logic I - [http://arxiv.org/abs/0805.3521v4] Towards applied theories based on computability logic - [http://arxiv.org/abs/1011.5687v1] Topological Modal Logics with Difference Modality - [http://arxiv.org/abs/2604.05712v1] Precise measurement of the CKM angle γ with a novel approach - [http://arxiv.org/abs/2605.11464v1] Study of $\varphi \to K\bar{K}$ in the amplitude analysis of $D^+ \to K^0_S K^0_L \pi^+$ - [http://arxiv.org/abs/2503.11383v3] Study of $\varphi \to K\bar{K}$ and $K^0_S - K^0_L$ Asymmetry in the Amplitude Analysis of $D_{s}^{+} \to K_{S}^{0} K_{L}^{0}$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.9s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(abs(x) != abs(y) for x, y in zip(clause, clause[1:])):
                clauses.append(clause)
        return clauses

    def resolution_width(cnf):
        n = len(cnf[0])
        clauses = set(tuple(sorted(c)) for c in cnf)
        queue = list(clauses)
        seen = set(queue)

        while queue:
            literal = random.choice([p for clause in queue for p in clause if p > 0])
            new_clauses = []
            for clause in queue:
                if -literal in clause:
                    continue
                new_clause = [p for p in clause if p != literal]
                if not new_clause:
```

```
                return len(queue)
            new_clause.sort()
            if tuple(new_clause) not in seen:
                seen.add(tuple(new_clause))
                new_clauses.append(new_clause)
        queue.extend(new_clauses)

    return len(queue)

def quantum_logic_rank(cnf):
    n = len(cnf[0])
    matrix = [[Fraction(1, 2)] * (n + 1) for _ in range(n + 1)]
    for clause in cnf:
        for p in clause:
            if p > 0:
                matrix[p - 1][n] += Fraction(1, len(clause))
            else:
                matrix[-p - 1][n] -= Fraction(1, len(clause))

    for i in range(n):
        if matrix[i][i] == 0:
            return None
        for j in range(n + 1):
            matrix[i][j] /= matrix[i][i]
        for k in range(n + 1):
            if k != i and matrix[k][i] != 0:
                factor = -matrix[k][i]
                for j in range(n + 1):
                    matrix[k][j] += factor * matrix[i][j]

    rank = n
    for i in range(n, -1, -1):
        if all(matrix[j][i] == 0 for j in range(n + 1)):
            rank -= 1

    return rank

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = generate_cnf(n)
    rank = quantum_logic_rank(cnf)
    width = resolution_width(cnf)

    if rank is not None:
        results.append((rank, width))

if len(results) < 30:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n for _, _ in results),
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }
```

```
    ranks, widths = zip(*results)
    mean_rank = sum(ranks) / len(ranks)
    mean_width = sum(widths) / len(widths)
    correlation_coefficient = sum((r - mean_rank) * (w - mean_width) for r, w in results) / (len(r

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n for _, _ in results),
        "conjecture_holds": correlation_coefficient >= 0.7 and all(correlation_coefficient >= 0.5
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    total_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        total_results.append(result)

    mean_value = sum(r["metric_value"] for r in total_results if r["metric_value"] is not None) /
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in total_results if r["met
    support_fraction = sum(1 for r in total_results if r["conjecture_holds"]) / len(total_results)

    if all(r["conjecture_holds"] for r in total_results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in total_results) and any(r["metric_value"] >= 0.5 fo
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.7\" first_failing_s
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition of demonstrating a correlation coefficient r ≥ 0.7 between the minimal rank and resolution proof width for at least 80% of the CNF instances. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	25683
2	preregistration	ollama_remote	glm4:latest	0	0	14581
3	novelty	ollama_remote	glm4:latest	0	0	8163
4	novelty	ollama_remote	glm4:latest	0	0	9997
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19495
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17870
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11954
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	138881
9	judge	ollama_remote	glm4:latest	0	0	35564

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 282189 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/559c6c08bf2f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/559c6c08bf2f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/559c6c08bf2f.tar.gz` (if generated)